

Portalizing Dropdowns: Notes From A Debugging Session

TL;DR: I discovered that whenever a parent element uses CSS transforms, native `<select>` dropdown menus can teleport to the top-left corner of the screen. Portal-based components (like `react-select`) or a hand-rolled `createPortal` menu kept my dropdowns anchored where users expect them.

Why I'm Sharing This

While wrapping up an admin dashboard, I noticed the role picker occasionally appeared in the corner of the screen—hardly the polished experience I wanted. I dug into the issue, tried a few fixes, and wrote everything down in case someone else trips over the same thing (or just wants to see how I think through UI bugs). What follows is a faithful retelling: how I reproduced the bug, what I learned, and the solutions that felt right for my situation.

Reproducing The Bug

1. Scaffold a Vite React app

```
npm create vite@latest warped-select-demo -- --template react-ts
cd warped-select-demo
npm install
```

2. Replace `src/App.tsx` with the snippet below. The outer card uses `transform: translateY(-24px)` —a common trick for animated dashboards.

```
import './App.css';

export default function App() {
  return (
    <div className="page">
      <div className="admin-card">
        <label htmlFor="role">Role</label>
        <select id="role" defaultValue="viewer">
          <option value="viewer">Viewer</option>
          <option value="editor">Editor</option>
          <option value="super_admin">Super Admin</option>
        </select>
      </div>
    </div>
  );
}
```

3. Use this `src/App.css` to mimic a scaled admin panel:

```
:root {
  font-family: 'Inter', system-ui, sans-serif;
  background: #0f172a;
  color: #f8fafc;
```

```

}

.page {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
}

.admin-card {
  width: 320px;
  padding: 1.5rem;
  border-radius: 14px;
  background: rgba(15, 23, 42, 0.8);
  backdrop-filter: blur(18px);
  transform: translateY(-24px);
}

select {
  width: 100%;
  margin-top: 0.5rem;
  padding: 0.65rem;
  border-radius: 8px;
}

```

4. Run `npm run dev` and open the dropdown. The menu snaps to the top-left corner of the screen instead of expanding under the field—exactly the bug that annoyed our users.

Root Cause

After some console spelunking and a few “why is this happening” searches, I realized CSS transforms create a new containing block and mess with how browsers compute `position: absolute` offsets. Native dropdowns render their menus in a compositor layer that ignores transformed ancestors, so the origin falls back to `(0, 0)` of the viewport. Any dashboard that leans on `transform`, `perspective`, or `filter` can run into the same surprise.

Solution 1: Drop-In Fix With `react-select`

My quickest win was swapping the native field for `react-select`. Because it renders the menu through a portal, the dropdown ignores any transformed parents and just works.

Install the dependency:

```
npm install react-select
```

I turned it into a small reusable field (`src/components/common/SelectField.tsx` in my project):

```

import Select from 'react-select';
import type { SingleValue } from 'react-select';

const ROLE_OPTIONS = [

```

```

    { value: 'viewer', label: 'Viewer – view only' },
    { value: 'editor', label: 'Editor – manage content' },
    { value: 'super_admin', label: 'Super Admin – full access' },
  ];

  interface Props {
    value: string;
    onChange: (value: string) => void;
    disabled?: boolean;
  }

  export default function RoleSelect({ value, onChange, disabled }: Props) {
    return (
      <Select
        inputId="role"
        classNamePrefix="admin-select"
        options={ROLE_OPTIONS}
        value={ROLE_OPTIONS.find((opt) => opt.value === value)}
        onChange={(option: SingleValue<typeof ROLE_OPTIONS[number]>) =>
          onChange(option?.value ?? 'viewer')}
        isDisabled={disabled}
        menuPortalTarget={document.body}
        menuPosition="fixed"
        styles={{
          menuPortal: (base) => ({ ...base, zIndex: 9999 }),
          control: (base) => ({ ...base, minHeight: 48, borderRadius: 8 }),
        }}
      />
    );
  }

```

Plugging it into the admin form (`src/pages/admin/UserForm.tsx`) looked like this:

```

<RoleSelect
  value={formData.role}
  disabled={loading || isEditingSelf}
  onChange={(role) => setFormData((prev) => ({ ...prev, role })))}
/>
{isEditingSelf && (
  <small style={{ color: '#e53e3e' }}>You cannot change your own role</small>
)}

```

Because the menu now lives directly under `document.body`, the role picker stayed put even when I kept the fancy dashboard transforms.

Solution 2: Custom Portal Menu Without Dependencies

In another branch I experimented with a no-dependency version. The idea is to render a button, measure it, and drop a manual menu into `document.body` via `createPortal`.

```

import { useRef, useState } from 'react';
import { createPortal } from 'react-dom';

const ROLE_OPTIONS = [
  { value: 'viewer', label: 'Viewer – view only' },
  { value: 'editor', label: 'Editor – manage content' },
  { value: 'super_admin', label: 'Super Admin – full access' },
];

export function PortalDropdown() {
  const buttonRef = useRef<HTMLButtonElement>(null);
  const [open, setOpen] = useState(false);
  const [role, setRole] = useState('viewer');

  const menuStyle = (() => {
    const rect = buttonRef.current?.getBoundingClientRect();
    if (!rect) return { display: 'none' } as const;
    return {
      position: 'fixed' as const,
      top: rect.bottom + 6,
      left: rect.left,
      width: rect.width,
      background: '#fff',
      color: '#1f2937',
      borderRadius: 10,
      boxShadow: '0 20px 40px rgba(0,0,0,0.16)',
      zIndex: 10000,
    };
  })();

  return (
    <>
      <button ref={buttonRef} className="pseudo-select" onClick={() => setOpen((v)
=> !v)}>
        {ROLE_OPTIONS.find((opt) => opt.value === role)?.label}
      </button>

      {open &&
        createPortal(
          <ul style={menuStyle} role="listbox">
            {ROLE_OPTIONS.map((opt) => (
              <li
                key={opt.value}
                onClick={() => {
                  setRole(opt.value);
                  setOpen(false);
                }}
                role="option"
                aria-selected={role === opt.value}
              >
                {opt.label}
            )}
          )}
        )}
    </>
  );
}

```

```

        </li>
      )}}
    </ul>,
    document.body,
  )}
</>
);
}

```

This kept the bundle lean and gave me full control over animations, accessibility, and theming when I needed it.

Solution 3: Remove The Transform Context

One last option I considered was questioning whether the transform was worth it at all. When it was just there for positioning, swapping it for `margin`, `top/left`, or a layout utility removed the problem entirely.

```

/* Before */
.admin-card {
  transform: translateY(-24px);
}

/* After */
.dashboard-panel {
  display: grid;
  place-items: center;
}

.admin-card {
  margin-top: -24px; /* visual offset without a transform */
}

```

Validation Checklist

- ☒ I ran `npm run lint` so TypeScript + ESLint would yell if I forgot to wire the new component.
- ☒ I tabbed through the form to confirm keyboard accessibility (`react-select` exposes `inputId` for screen readers).
- ☒ I tested the dropdown inside modals/drawers and tweaked `menuPortalTarget` / `z-index` so it layered correctly.
- ☒ I recorded before/after clips for LinkedIn/Medium because the contrast tells the story way better than words.

Final Thoughts

I'm sharing this partly so future-me won't forget, and partly because little UX paper cuts like this tend to pop up at the worst time. If you run into dropdowns that teleport when a container is transformed, hopefully one of these experiments saves you an afternoon. If you try something cleaner, I'd love to hear about it.